

UNIVERSIDADE ESTADUAL DO CEARÁ

Curso de Ciência da Computação

Lista de Exercícios 01 - ED

Arrays, Structs, Ponteiros e Organização de Memória em C

Prof. Dr. Bonfim Amaro Junior

Instruções:

- Justifique suas respostas sempre que necessário.
- Utilize linguagem C padrão.
- Considere esse material para seu estudo dos elementos de programação na linguagem C.

Q1. Considere o código abaixo:

```
int v[5] = {1,2,3,4,5};  
printf("%p %p %p \n", &v[0], &v[1], &v[2]);
```

Explique o comportamento dos endereços impressos e sua relação com o tamanho do tipo de dado.

Q2. Implemente uma função que receba um vetor de inteiros e seu tamanho, e retorne o maior elemento presente no vetor.

A implementação **deve obrigatoriamente utilizar aritmética de ponteiros**, ou seja, **não é permitido o uso do operador de indexação** (por exemplo, `v[i]`).

Assinatura esperada da função:

```
int max(int *v, int n);
```

Requisitos:

- O vetor deve ser percorrido utilizando apenas ponteiros (ex.: `*(v + i)` ou incremento de ponteiros).
- A função deve funcionar para qualquer tamanho $n > 0$.
- Não utilizar variáveis auxiliares desnecessárias.

Exemplo: Se $v = \{3, 7, 2, 9, 5\}$, então a função deve retornar 9.

Q3. Considere o seguinte programa em C:

```
#include <stdio.h>

void f(int v[]) {
    printf("Dentro de f:\n");
    printf("sizeof(v)=%zu\n", sizeof(v));
}

int main() {
    int v[10];

    printf("No main:\n");
    printf("sizeof(v)=%zu\n", sizeof(v));

    f(v);

    return 0;
}
```

Pergunta:

Explique detalhadamente a saída do programa.

Em sua resposta, você deve:

- indicar os valores esperados de `sizeof(v)` no `main` e na função `f`;
- explicar por que os valores são diferentes;
- descrever o que acontece com o array quando ele é passado como parâmetro de função;
- relacionar esse comportamento com o conceito de **decaimento de array para ponteiro** (array decay);
- discutir por que esse comportamento pode gerar erros em funções que manipulam vetores.

- Como as áreas de memória de um processo em C (STACK x HEAP) atuam

Considere uma arquitetura onde:

- `sizeof(int)` = 4 bytes;
- ponteiros ocupam 8 bytes.

Q4. Implemente um programa em C para manipular dinamicamente um vetor de inteiros cujo tamanho é informado apenas em tempo de execução.

O programa deve ser estruturado de forma modular e conter, obrigatoriamente, pelo menos as seguintes etapas:

- a) Ler um valor inteiro positivo n , correspondente à quantidade de elementos do vetor;
- b) Alocar dinamicamente memória suficiente para armazenar n elementos do tipo `int`;
- c) Ler os n valores do vetor;
- d) Calcular e informar:
 - o maior valor armazenado;
 - o menor valor armazenado;
 - a soma de todos os elementos;
 - a média aritmética dos elementos.
- e) Liberar corretamente toda a memória alocada antes do encerramento do programa.

Requisitos obrigatórios:

- A alocação deve ser feita com `malloc` ou `calloc`;
- O retorno da função de alocação deve ser verificado. Caso a alocação falhe, o programa deve exibir uma mensagem de erro apropriada e encerrar a execução de forma segura;
- O vetor deve ser manipulado por meio de ponteiros ou passagem de vetor para funções;
- A solução deve ser organizada em funções, evitando concentrar toda a lógica na função `main`;
- A memória alocada deve ser liberada com `free`, e o ponteiro deve ser invalidado após a liberação;
- Não é permitido assumir um tamanho fixo para o vetor em tempo de compilação.

Assinatura mínima esperada para uma das funções:

```
int *aloca_vetor(int n);
```

Você pode definir outras funções auxiliares, como por exemplo:

```
void ler_vetor(int *v, int n);  
void imprimir_estatisticas(int *v, int n);
```

Observação: respostas que apenas alocam, leem e imprimem os elementos do vetor, sem modularização e sem tratamento de erro, não atendem integralmente ao que foi solicitado.

Q5. Implemente uma função que inverta os elementos de um vetor de inteiros **in-place**, ou seja, sem utilizar memória auxiliar proporcional ao tamanho do vetor.

A função deve receber um ponteiro para o vetor e o número de elementos, e realizar a inversão diretamente na região de memória original.

Assinatura esperada:

```
void inverter(int *v, int n);
```

Requisitos obrigatórios:

- A solução deve utilizar **aritmética de ponteiros** (não utilizar `v[i]`);
- Não é permitido utilizar vetores auxiliares;
- A função deve funcionar corretamente para:
 - vetor com 1 elemento;
 - vetor com tamanho par;
 - vetor com tamanho ímpar;

Exemplo:

Entrada:

```
v = {1, 2, 3, 4, 5}
```

Saída esperada:

```
v = {5, 4, 3, 2, 1}
```

Além da implementação, explique:

- por que a solução não necessita de memória adicional significativa;
- como os ponteiros devem ser movimentados durante a execução;

Dica: considere o uso de dois ponteiros que percorrem o vetor a partir de extremidades opostas.

Q6. Considere o seguinte trecho de código em C:

```
#include <stdio.h>

int main() {
    const int v[3] = {10, 20, 30};
    const int *p = v;

    printf("%d\n", *(p + 1));
    printf("%d\n", p[2]);
    printf("%d\n", *(v + 0));

    return 0;
}
```

- Indique os valores que serão impressos pelo programa.
- Explique detalhadamente como cada uma das expressões acessa os elementos do vetor:
 - `*(p + 1)`
 - `p[2]`
 - `*(v + 0)`
- Mostre a equivalência formal entre indexação e aritmética de ponteiros em C.
- Considerando que `sizeof(int) = 4`, explique como o endereço acessado em `*(p + 1)` é calculado.
- Discuta por que a linguagem C permite tratar *arrays* e ponteiros de forma semelhante, mas não idêntica.

Q7. Considere a seguinte implementação em C:

```
void inicializa(int *v, int n) {
    int *p = v;

    while (p <= v + n) {
        *p = 0;
        p++;
    }
}
```

```
}  
}
```

- a) Explique qual é o objetivo da função apresentada.
- b) Identifique o erro presente na implementação. Seja preciso ao indicar:
 - a linha problemática;
 - o tipo de erro cometido;
 - por que esse erro ocorre.
- c) Descreva quais podem ser as consequências desse erro durante a execução do programa.
- d) Reescreva a função corrigindo o problema, mantendo o uso exclusivo de aritmética de ponteiros.

Q8. Considere o código implementado na linguagem C com a função "**int** calcular" incompleta:

```
#include <stdio.h>  
  
int soma(int a, int b) { return a + b; }  
int subtrai(int a, int b) { return a - b; }  
int multiplica(int a, int b) { return a * b; }  
  
int calcular( ...  
  
int main() {  
  
    int a, b, opcao;  
  
    printf("Digite dois numeros: ");  
    scanf("%d %d", &a, &b);  
  
    printf("\n1 - Soma\n")  
    printf("2 - Subtracao\n")  
    printf("3 - Multiplicacao\n")  
    printf("Escolha: ");  
    scanf("%d", &opcao);  
  
    int resultado;  
  
    if (opcao == 1)  
        resultado = calcular(a, b, soma);
```

```

        else if (opcao == 2)
            resultado = calcular(a, b, subtrai);
        else if (opcao == 3)
            resultado = calcular(a, b, multiplica);
        else {
            printf("Opcao invalida!\n");
            return 1;
        }

        printf("Resultado: %d\n", resultado);

        return 0;
}

```

Complete a assinatura da função calcular para receber dois inteiros e uma função que tem como entrada dois inteiros e que retorna um inteiro como saída. Objetivo é realizar o calculo de acordo com a função passada por parâmetro (soma, subtrai ou multiplica).

Q9. Implemente uma função que recebe dois vetores ordenados e retorna um novo vetor ordenado contendo a fusão (merge), utilizando alocação dinâmica.

Requisitos:

- uso correto de malloc e free;
- evitar vazamento de memória.

Q10. Considere:

```

int *f() {
    int x = 10;
    return &x;
}

```

Explique detalhadamente o erro. Discuta:

- tempo de vida da variável;
- região de memória envolvida;
- consequências práticas.

Q11. Implemente uma função que retorna um vetor alocado dinamicamente contendo os números pares até n.

Q12. Considere a seguinte struct:

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};
```

Implemente uma função que receba um vetor de `struct Aluno` e retorne um ponteiro para o aluno com maior nota.

Requisitos:

- utilizar aritmética de ponteiros (não usar indexação);
- não criar cópia do aluno;
- tratar o caso de vetor vazio.

Q13. Implemente uma função que receba um vetor dinâmico de `struct Aluno` e:

- aumente a nota de todos os alunos em 10%;
- limite a nota máxima em 10.0.

A função deve modificar diretamente o vetor original.

Explique:

- por que a passagem por ponteiro permite modificar os dados;
- o que ocorreria se a struct fosse passada por valor.

Q14. Implemente uma função que remova um aluno de um vetor dinâmico de structs, dado o índice do elemento.

Requisitos:

- realocar o vetor com `realloc`;
- manter os elementos restantes contíguos;
- atualizar corretamente o tamanho do vetor;
- evitar vazamento de memória.

Assinatura sugerida:

```
int remove_aluno(struct Aluno **v, int *n, int idx);
```


Q15. (Questão Bônus – Representação e Manipulação de Estruturas em Memória)

Considere a seguinte estrutura:

```
struct Vetor {  
    int *dados;  
    int tamanho;  
};
```

Deseja-se implementar uma função que receba um vetor dinâmico e produza um novo vetor contendo os mesmos elementos, porém em ordem invertida.

Tarefa principal:

Implemente a função:

```
struct Vetor inverter_copia(const struct Vetor *v);
```

Essa função deve:

- alocar dinamicamente um novo vetor;
- copiar os elementos do vetor original em ordem inversa;
- retornar uma nova estrutura `Vetor` contendo o resultado;
- não modificar o vetor original.

Requisitos obrigatórios:

- utilizar corretamente `malloc` e `free`;
- verificar falha de alocação;
- acessar os elementos utilizando aritmética de ponteiros;
- garantir que não haja vazamento de memória.

Parte conceitual (obrigatória):

Explique detalhadamente:

- a) a diferença entre copiar um vetor e apenas copiar o ponteiro (*cópia superficial vs cópia profunda*);
- b) por que a função não deve retornar apenas um ponteiro para o vetor original invertido;
- c) qual é a região de memória onde o novo vetor será alocado e qual seu tempo de vida;
- d) o que poderia ocorrer caso a função retornasse um ponteiro para uma variável local;

Parte avançada:

Discuta como a função deveria ser modificada caso:

- o vetor fosse muito grande e a cópia completa fosse inviável;
- fosse desejado evitar alocação adicional de memória.